

C89/C90 (ANSI C)



Quick Reference

Block-scope variables · Storage-class specifiers

· Global variables · Implicit conversions

· Structs · Pointers & arrays

C90 reference · Arthur

What is C90 / ANSI C / ISO C 90?

- **C90 (ANSI C89):** first standardized C; portability baseline many toolchains still support.
- **Pre-C99 limits:** declarations at block start; no `//` comments (use `/*...*/`).
- **Linkage & storage:** file-scope names are external by default; **static** at file scope $\Rightarrow$ internal; globals/statics zero-initialized.
- **Zero-init:** globals/statics are zero-initialized by default.

Declarations at Block Start (C90)

- **C90: Declare variables at the start of each block (no mixed decls/statements).**

```
#include <stddef.h>
long sum_ints(const int *a, size_t n)
{
    size_t i;      /* C90: declarations at block start */
    long s = 0;

    for (i = 0; i < n; ++i)
        s += a[i];
    return s;
}
```


Uninitialized Locals $\Rightarrow$ UB (C90)

- Locals without an initializer have indeterminate value.
- Read before assign $\rightarrow$ UB (-O2 -Wall warns).
- Fix: initialize at declaration or assign on all paths.
- Often reads stale stack bytes (“garbage”) still UB.

```
int f(int c)
{
    int x;          /* indeterminate */
    if (c > 0) x = 5;
    return x;       /* UB when c <= 0 (may warn) */
}
```

Storage in C90: auto · static · dynamic

- **auto** (block default): automatic storage; lives for the block.
- **static** (in block): static storage; zero-init. At file scope: internal linkage.
- **dynamic** (heap): Dynamic: malloc → indeterminate bytes.

```
#include <stddef.h>
#include <stdlib.h>
void storage_demo(size_t n)
{
    size_t i;
    int *p = malloc(n * sizeof *p);
    int local = 0;    /* auto */
    static long hits; /* static: persists, zero-initialized */

    ++hits; /* survives across calls */
    if (p) { for (i = 0; i < n; ++i) p[i] = 0; free(p); p = NULL; }
    (void)local; /*UB warn fix*/
}
```

Global Variables (C90)

- File scope → static storage; no init → zero-init.
- Static-storage init: constant expr or string literal.
- Linkage: default external; file-scope static = internal.
- C quirk: file-scope const is external (not like C++).
- Tentative def: `int g;` becomes real def (`=0`) if no other def appears; use `extern` in headers.

```
/* globals.h */
extern int g;

/* globals.c */
int g = 0; /* single definition */

/* use.c */
void f(void){ g++; }

/* file-local “global” */
static int counter; /* internal linkage, zero-init*/
void inc(void){ ++counter; }
```


Implicit conversions (C90)

- Where: assignment/init, operators, calls (with prototype).
- Integer promotions: char/short/enum → int (or unsigned int).
- Usual arithmetic conversions: find a common type; signed+unsigned may become unsigned.
- Float promotions: float → double in expressions and in variadic / no-prototype calls.
- Pointers: void* converts to/from any object pointer without a cast in C (not function pointers; C++ differs).
- Null pointer constants: 0/NULL become the proper null pointer value.
- File-scope init: must be a constant expression or string literal (not another variable).

```
#include <stddef.h>
/* File scope (static storage) */
double Gd = (double)42; /* cast on constant OK */
int *Gp = NULL; /* null pointer */
/*int A = (int)Gd; NOT allowed at file scope: not a constant init */

int demo(unsigned u, short s, float f, void *vp)
{
    double t = u + s + f; /* implicit casts result as double*/
    int *pi = vp; /* void*->int* (object pointer; cast not needed in C) */
    return (int)t + (pi ? *pi : 0); /* explicit cast on return */
}
```

Comments in C (C90)

- C90 comments: only `/* ... */`. `//` is C99+ (errors in strict C90).
- No nesting: `/* ... /* ... */ ... */` is invalid.
- Commenting out code: prefer `#if 0 ... #endif` when the block already contains `/* ... */`.
- Enforce it: compile with `-std=iso9899:1990` (or `:199409`) + `-pedantic` to reject extensions like `//`.

```
/* Good (C90): */
```

```
int x = 0; /* explain... */
```

```
/* Bad: nested comment (illegal in C90) */
```

```
/* start /* inner */ end */
```

```
/* Strings are not comments: */
```

```
printf("literal /* not a comment *\n");
```

```
/* Tip: keep blocks short; don't "comment-out" big code with /*...*/  
if it already has comments. */
```


structs in C90

- **struct**: user-defined aggregate of named members; stored in declared order; padding/alignment impl-defined.
- **Define/use**: `struct T { ... }; struct T v;` self-reference via pointer only (`struct Node *next;`).
- **Init (C90)**: brace list in member order; no designated inits / no compound literals (C99+).
- **Copying**: whole-struct assignment and pass/return by value; copy is shallow.
- **Bit-fields (C90)**: only `int` / `unsigned int` / `signed int`; layout impl-defined.
- **offsetof**: `offsetof(T, member)` from `<stddef.h>` → byte offset (not for bit-fields).

```
#include <stddef.h>
struct Point { int x, y; };      /* a type that bundles two variables */
struct Node { int v; struct Node *next; }; /* self via pointer */
struct Flags { unsigned int a:1, b:1; }; /* C90 bit-fields */

static struct Point G = { 1, 2 }; /* file-scope init: const expr */

struct Point demo(struct Point p)
{
    struct Point q = { 3, 4 }; /* positional init (C90) */
    q = p;                    /* struct shallow copy */
    size_t off = offsetof(struct Point, y);
    (void)off;

    return q;                  /* return by value */
}
```

Pointers & arrays (C90)

- Array params: `int a[]` is adjusted to `int *a`; pass length `n`.
- Subscript: `p[i] == *(p + i)`; steps by `sizeof *p`.
- Ptr arithmetic: Only within the same array; “one-past” OK, don’t deref.
- Deref safety: `*p` needs a valid, aligned object pointer; null/dangling $\Rightarrow$ UB.
- `void*`: Implicit to/from any object pointer in C (not functions).
- Null pointer: `0`/NULL convert to the proper null pointer value.
- Swap: Use a temp; XOR-swap breaks when operands alias / same.
- Overlap: Don’t `memcpy` overlapping ranges; use `memmove`.

```
void swap_int(int *a, int *b) {  
    int temp;  
    if (!a || !b) return;  
    if (a==b) return; /* optional check */  
    temp = *a; *a = *b; *b = temp;  
}
```

```
#include <stddef.h>  
#include <stdlib.h>  
int pointers_demo(void) {  
    int arr[5] = {1,2,3,4,5};  
    size_t n = sizeof arr / sizeof arr[0];  
    int *p = arr;          /* array  $\rightarrow$  pointer decay */  
    int *q = arr + 3;       /* pointer arithmetic */  
    int first = p[0];       /* == *p */  
    int fourth = *(p + 3);  /* == arr[3] */  
    int *end = arr + n;     /* one-past — do NOT deref */  
  
    swap_int(&arr[1], &arr[3]); /* OK */  
    swap_int(&arr[2], &arr[2]); /* same pointer — still OK */  
    return first + fourth + (end > p);  
}
```

What's next after C90

quick roadmap

- POSIX threads (pthreads): create/join threads; sync with mutexes & condvars.
- RTOS fundamentals: tasks, priorities, preemption, ISRs, timing.
[Watch Digi-Key's "Intro to RTOS" series.](#)
- FreeRTOS hands-on: build a two-task demo; use queues (xQueueSendFromISR) for ISR→task communication.

